💻 GovFlow AI: System Architecture for Citizen Service Guidance in Sri Lanka

GovFlow AI — System Architecture

Version: 2.0

Date: July 2026

Status: Architecture Review

Country: Sri Lanka

Author: GovFlow AI Team

Table of Contents

1. Executive Summary
2. Problem Statement
3. Solution Overview
4. High-Level System Architecture
5. User Journeys & Data Flow
6. Core Modules
7. Hybrid AI + Rule Engine Design
8. Database Architecture
9. API Design
10. Security
11. Deployment
12. Technology Stack
13. MVP Roadmap
14. Non-Functional Requirements
15. Design Decisions

---

1. Executive Summary

GovFlow AI is a **chat-first citizen service guidance platform** for Sri Lanka. A citizen describes what they want to do in plain language and receives a personalized, step-by-step government service workflow — with required documents, fees, official office locations, and verified source links.

> ### ℹ️ Core Architectural Principle
>
> **AI handles conversation. Verified rules handle procedure.**
>
> Workflow outputs are always generated from official, admin-reviewed data — never from LLM inference alone. This makes every output traceable, consistent, and auditable.

Key Outcomes

Outcome	Detail
Accurate guidance	Citizens receive correct workflows based on their specific situation
Admin-controlled content	Service data managed without developer involvement
Source-traceable	Every workflow step links to an official government source
AI-resilient	System functions even if the AI layer is unavailable

---

2. Problem Statement

Current State

Sri Lankan citizens navigating government services face:

- **No unified source** — each department maintains separate, inconsistent websites
- **Eligibility confusion** — requirements differ by age, district, application type, citizenship status
- **Document uncertainty** — citizens arrive unprepared, causing repeated trips
- **Language barriers** — official information is rarely available in Sinhala or Tamil
- **No progress tracking** — citizens manage complex multi-step processes on their own

Impact

Problem	Citizen Impact
Wrong document list	Wasted travel, rejected applications

Outdated fee info	Refused payments, unnecessary delays
No edge-case guidance	Confusion, repeated visits, escalations
Language gaps	Exclusion of non-English speaking citizens

---

3. Solution Overview

GovFlow AI is built on three pillars:

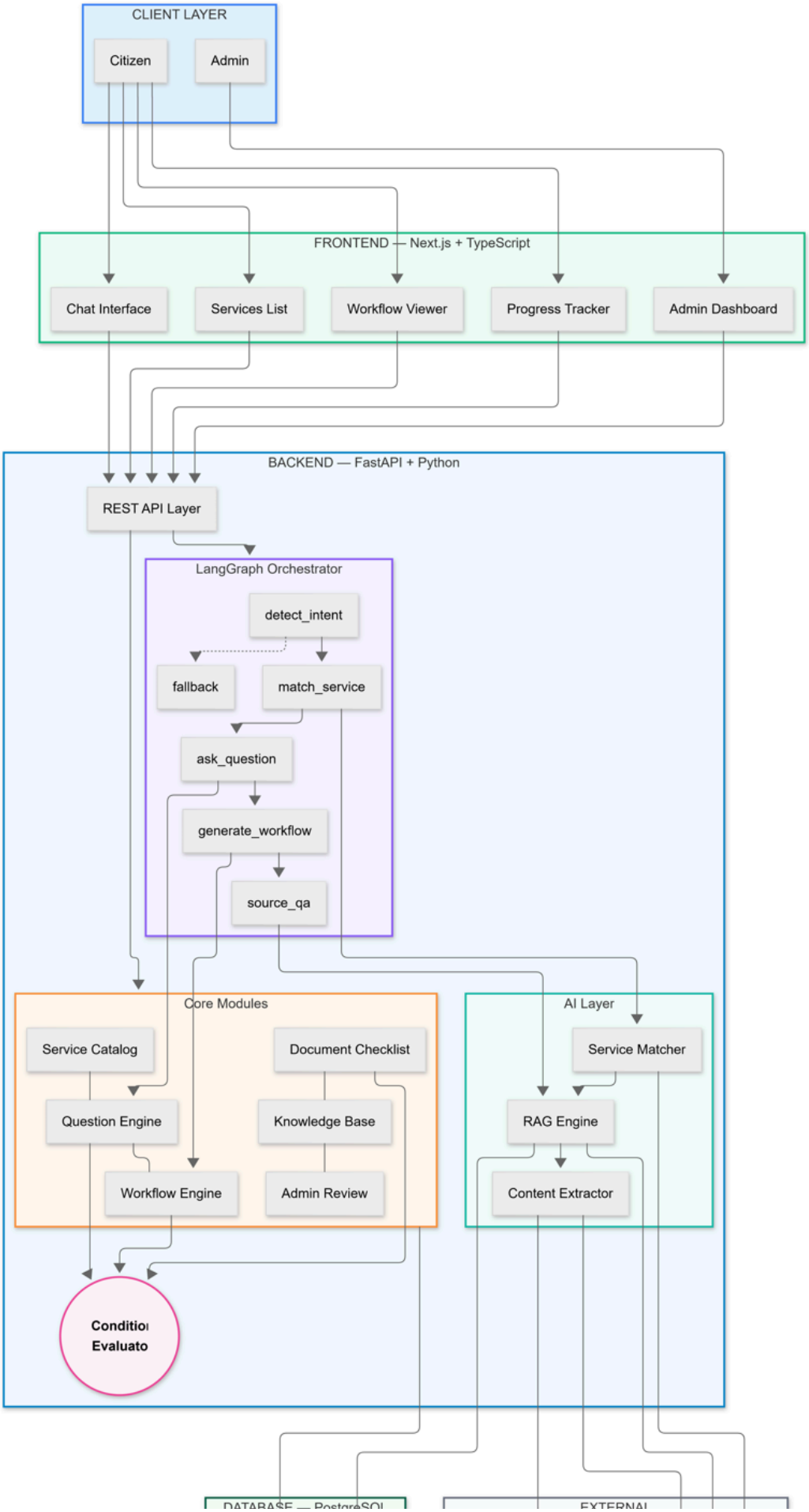
Pillar	What It Does
Chat Interface	Accepts natural language, asks targeted questions, returns a personalized result
Workflow Engine	Generates correct official workflows using verified rules — no AI guessing
Admin Platform	Allows content managers to add, review, and publish service data with AI assistance

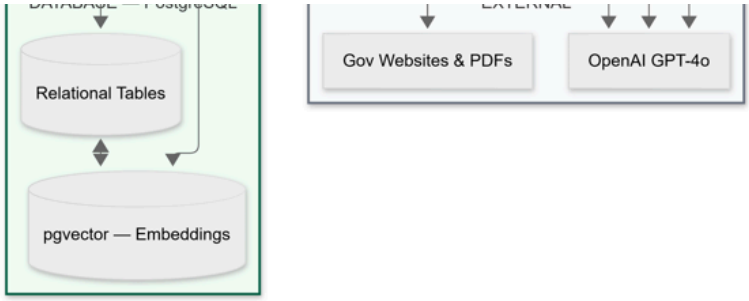
End-to-End Citizen Experience

```
Step 1   Citizen opens chat: "I want to renew my passport"
Step 2   AI understands intent → matches "Passport Renewal" service
Step 3   System asks clarifying questions (age, location, application type)
Step 4   Rule engine evaluates answers against stored conditions
Step 5   Citizen receives a personalized workflow:
           ✓ Steps in the correct order
           ✓ Required documents (with conditions explained)
           ✓ Official fees
           ✓ Office locations
           ✓ Official source links
Step 6   Citizen saves progress and marks steps as completed
Step 7   Citizen can ask follow-up questions (answered using official sources)
```

---

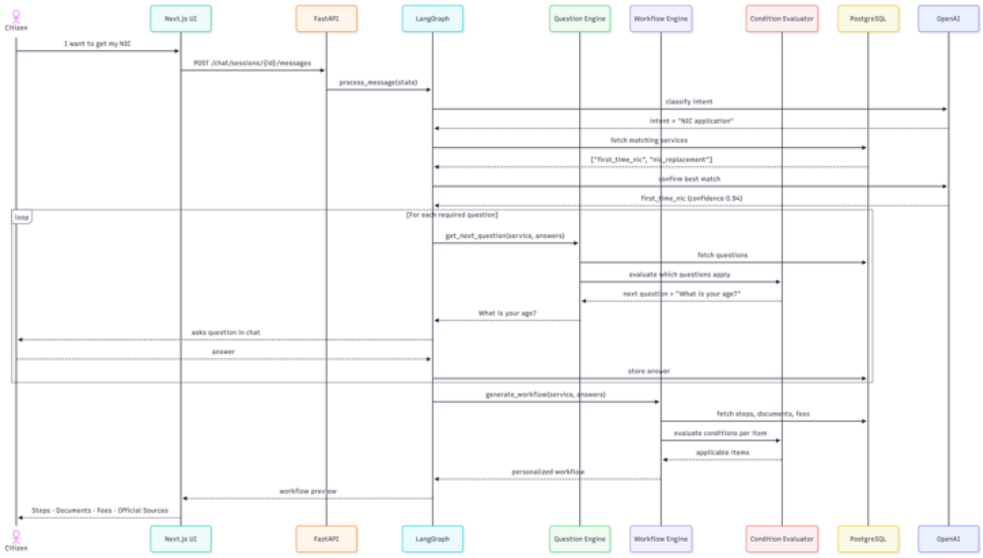
4. High-Level System Architecture


CLIENT LAYER
Citizen
Admin
FRONTEND — Next.js + TypeScript
Chat Interface
Services List
Workflow Viewer
Progress Tracker
Admin Dashboard
BACKEND — FastAPI + Python
REST API Layer
LangGraph Orchestrator
detect_intent
fallback
match_service
ask_question
generate_workflow
source_qa
Core Modules
Service Catalog
Document Checklist
Question Engine
Knowledge Base
Workflow Engine
Admin Review
AI Layer
Service Matcher
RAG Engine
Content Extractor
Conditio
Evaluato

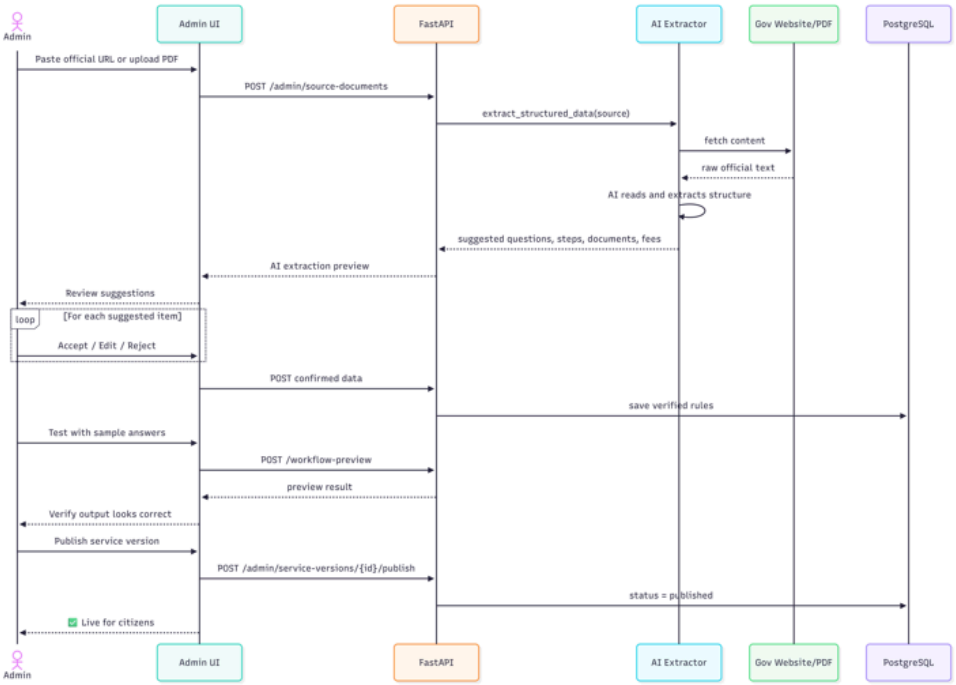


5. User Journeys & Data Flow

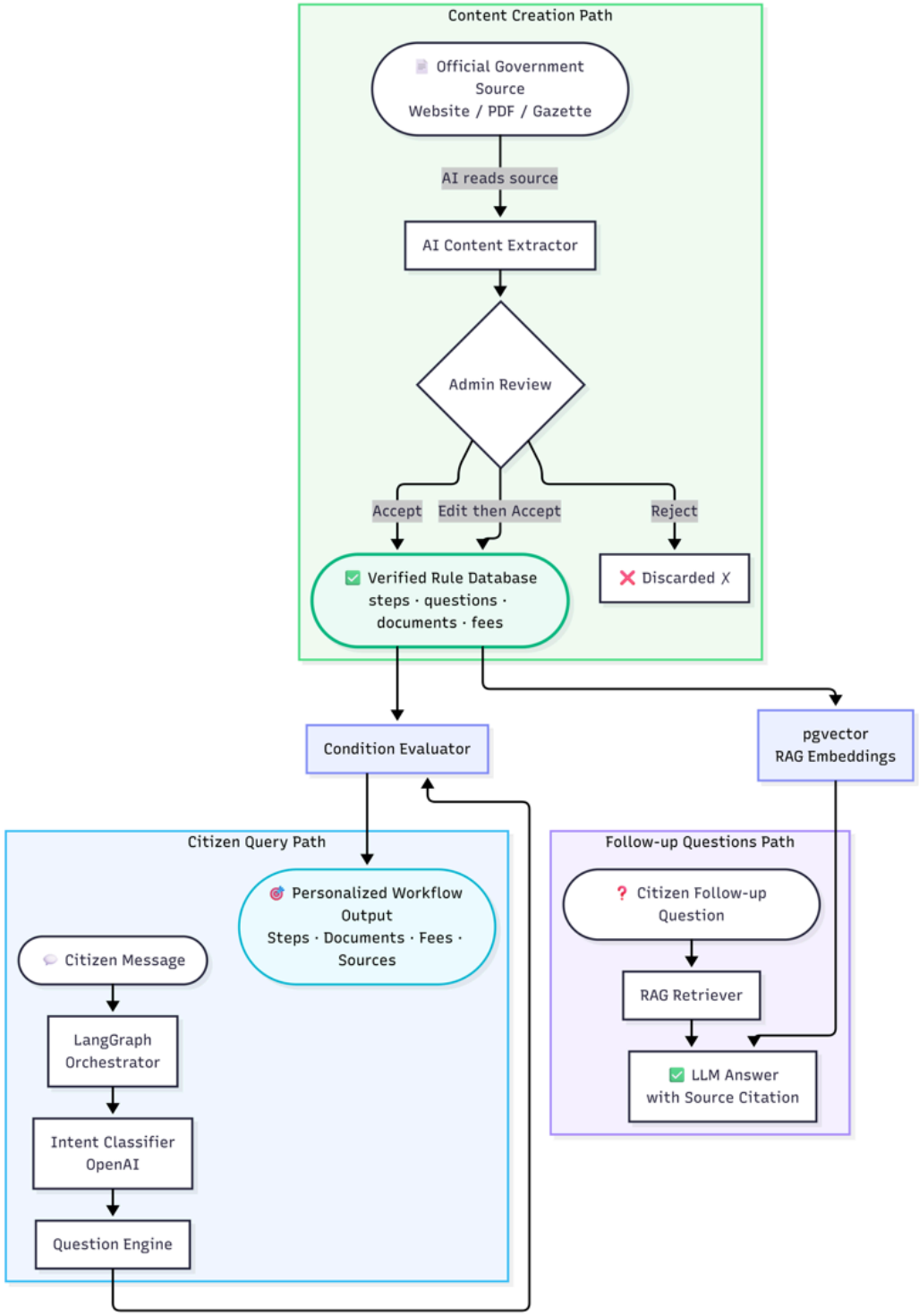
5.1 Citizen Chat Flow



5.2 Admin Content Creation Flow



5.3 LangGraph Node Flow



---

6. Core Modules

Module Summary

Module	Responsibility

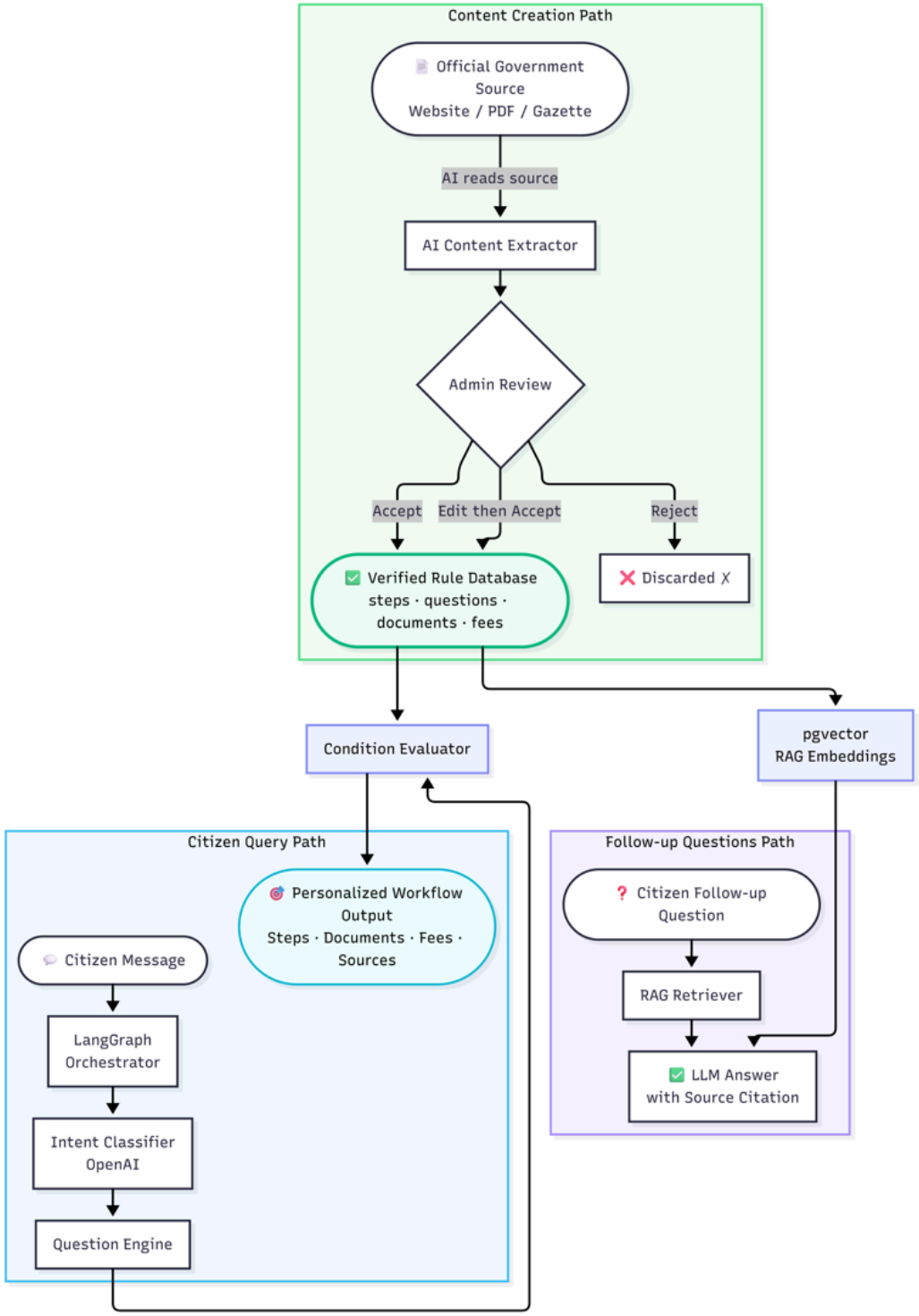
LangGraph Orchestrator	Manages conversation state from first message to saved workflow
Service Catalog	Stores and serves published government service definitions
Department Directory	Stores department contact details, addresses, official websites
Question Engine	Loads and sequences follow-up questions; evaluates which questions apply
Condition Evaluator	Shared rule evaluator used by questions, steps, documents, and fees
Workflow Engine	Generates personalized workflows by evaluating rules against user answers
Document Checklist	Determines required, optional, and conditional documents per workflow
Knowledge Base	Stores official source URLs, extracted text, and RAG embeddings
Progress Tracker	Stores and displays step completion and document readiness per user
Admin Review	Controls content creation, version management, and publishing
AI / RAG Module	Assists with intent matching, follow-up Q&A, and admin content extraction

---

7. Hybrid AI + Rule Engine Design

> **⚠️ Key Design Decision**
>
> A pure LLM (RAG-only) approach is **not used** for generating workflows. LLMs can hallucinate fees, invent requirements, and produce inconsistent answers. For a government-facing platform, this is unacceptable. Instead, we use a **hybrid model**: AI assists humans in creating rules; rules generate workflows deterministically.

The Hybrid Pipeline



AI Responsibility Boundaries

✅ AI Is Allowed To	❌ AI Is NOT Allowed To

Understand citizen's natural language intent	Invent fees or legal requirements
Match citizen to a government service	Decide final eligibility
Route the conversation through nodes	Publish content without admin approval
Explain published workflow steps in plain language	Override the rule engine output
Answer follow-up questions using official source chunks	Answer without citing sources
Extract structure from official docs (for admin review)	Modify workflow rules
Translate admin-reviewed content	Act as the authority on government procedure

Condition Rule Format

Every workflow step, question, document requirement, and fee has a `condition_json` field. The Condition Evaluator applies these rules to the citizen's answers.

Example — Include step only if applicant is an adult overseas applicant:

```
{
  "all": [
    { "field": "age", "operator": "greater_than", "value": 15 },
    { "field": "location", "operator": "equals", "value": "overseas" }
  ]
}
```

Example — Include if either condition matches:

```
{
  "any": [
    { "field": "application_type", "operator": "equals", "value": "lost" },
    { "field": "application_type", "operator": "equals", "value": "damaged" }
  ]
}
```

Supported Operators

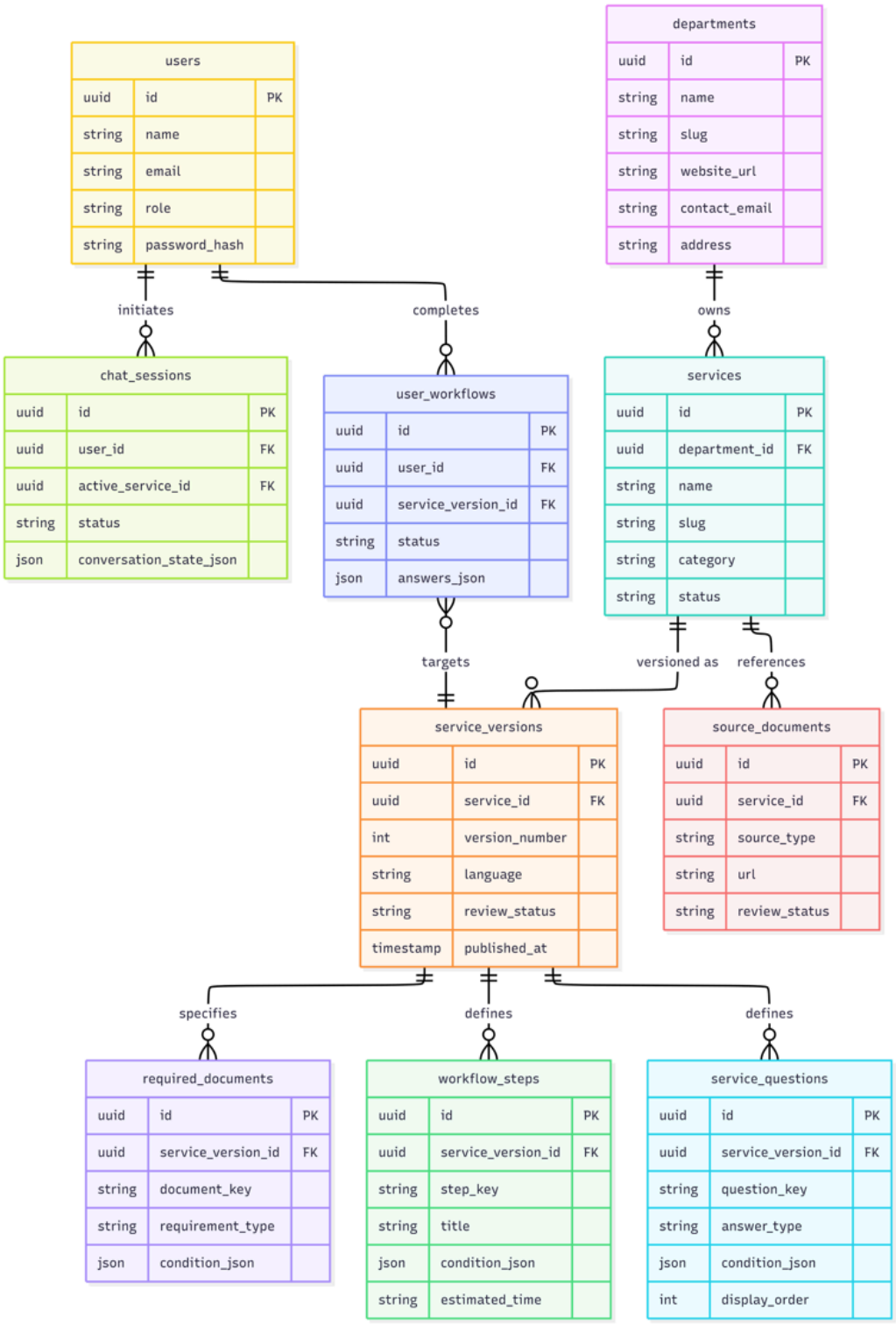
Operator	Usage
`equals`	Exact value match

`not_equals`	Value must not match
`in`	Value is in a list
`not_in`	Value is not in a list
`greater_than`	Numeric comparison
`less_than`	Numeric comparison
`between`	Numeric range (inclusive)
`exists`	Field has a value
`not_exists`	Field has no value

---

8. Database Architecture

8.1 Core Table Relationships



8.2 Core Tables

Table	Purpose

`departments`	Government department details, contact info, official website
`services`	Government service definitions with status lifecycle
`service_versions`	Versioned, reviewable content snapshots of a service
`service_questions`	Follow-up questions with display conditions
`question_options`	Answer options for choice-type questions
`workflow_steps`	Possible steps for a service with rule conditions
`required_documents`	Documents required by a service with conditions
`step_documents`	Maps workflow steps to their required documents
`fees`	Official or estimated fees with conditions
`processing_times`	Expected processing durations with conditions
`source_documents`	Official URLs, PDFs, and extracted text linked to services
`users`	Citizen and admin accounts with role
`chat_sessions`	Active conversations with LangGraph state
`chat_messages`	Individual messages in each chat session
`user_workflows`	Saved personalized workflows per citizen

`user_workflow_steps`	Per-user step completion tracking
`user_document_checklist`	Per-user document readiness tracking
`user_feedback`	Citizen-submitted corrections and issues
`audit_logs`	All admin actions with before/after state

8.3 Service Version Lifecycle

```
draft  →  in_review  →  approved  →  published
                    ↘  rejected  →  draft (revised)

archived  (when a newer version supersedes)
```

> ℹ️ **Important:** Citizens only ever see services in `published` status. No content goes live without admin review and explicit publish action.

---

9. API Design

Base URL: `/api/v1`

9.1 Citizen APIs

Method	Endpoint	Description
`POST`	`/chat/sessions`	Start a new chat session
`GET`	`/chat/sessions/{id}`	Get session and current state
`POST`	`/chat/sessions/{id}/messages`	Send a citizen message
`GET`	`/chat/sessions/{id}/messages`	Retrieve chat history
`GET`	`/services`	List all published services

`GET`	`/services/{slug}`	Get service detail
`POST`	`/workflow-preview`	Generate a workflow preview (no save)
`POST`	`/workflows`	Save a generated workflow
`GET`	`/workflows/{id}`	Retrieve a saved workflow
`PATCH`	`/workflows/{id}/steps/{step_id}`	Update step completion status
`PATCH`	`/workflows/{id}/documents/{doc_id}`	Update document readiness
`POST`	`/feedback`	Submit citizen feedback

9.2 Auth APIs

Method	Endpoint	Description
`POST`	`/auth/register`	Register new citizen account
`POST`	`/auth/login`	Login and receive JWT
`POST`	`/auth/logout`	Invalidate session
`GET`	`/auth/me`	Get current user profile
`POST`	`/auth/forgot-password`	Send password reset email
`POST`	`/auth/reset-password`	Set new password via token

9.3 Admin APIs

Method	Endpoint	Description

`GET/POST`	`/admin/departments`	List or create departments
`PATCH`	`/admin/departments/{id}`	Update department
`GET/POST`	`/admin/services`	List or create services
`PATCH`	`/admin/services/{id}`	Update service
`GET/POST`	`/admin/services/{id}/versions`	List or create service versions
`PATCH`	`/admin/service-versions/{id}`	Update version content
`POST`	`/admin/service-versions/{id}/publish`	Publish a version
`GET/POST`	`/admin/source-documents`	Manage official source links
`GET/POST`	`/admin/service-versions/{id}/questions`	Manage questions
`GET/POST`	`/admin/service-versions/{id}/workflow-steps`	Manage workflow steps
`GET/POST`	`/admin/service-versions/{id}/documents`	Manage document requirements
`GET`	`/admin/feedback`	View citizen feedback
`PATCH`	`/admin/feedback/{id}`	Update feedback status

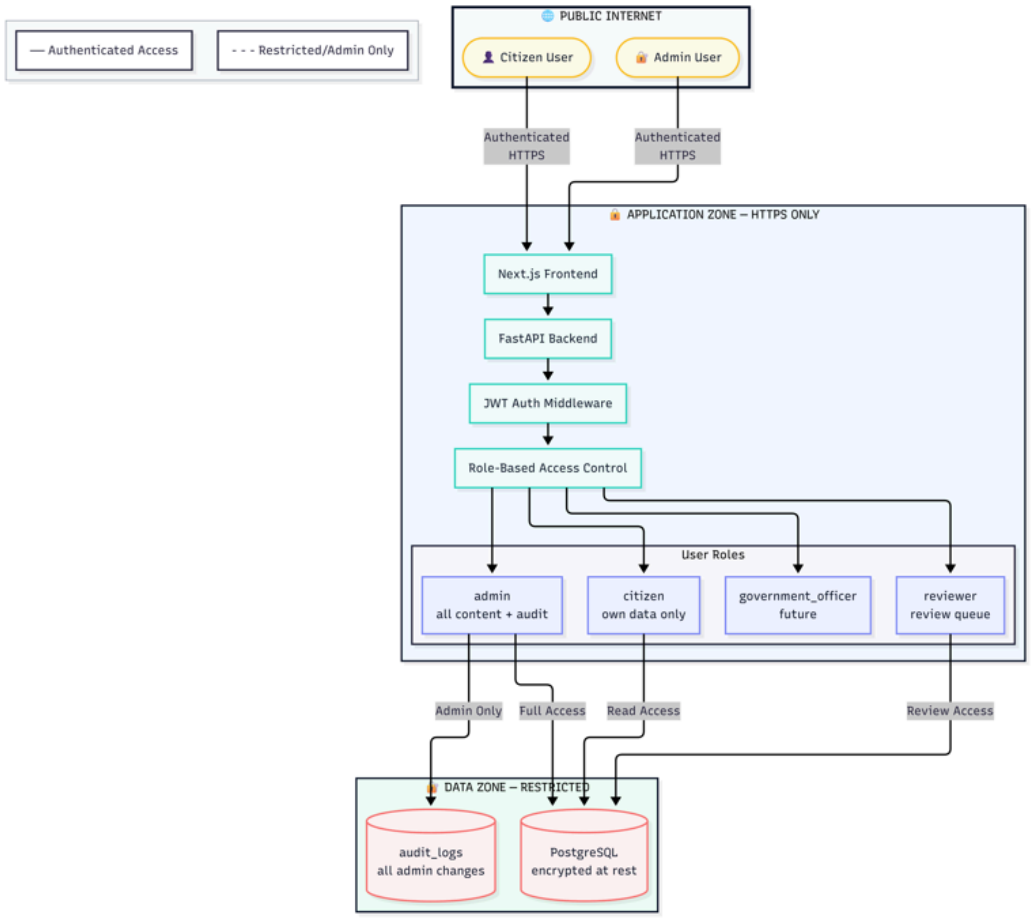
9.4 Internal AI APIs

Method	Endpoint	Description
`POST`	`/ai/match-service`	Match citizen intent to a service
`POST`	`/ai/ask`	Source-grounded Q&A using RAG
`POST`	`/ai/extract-from-source`	Extract structured data from official doc

---

10. Security

Access Control Model



Security Requirements

Category	Requirement

Transport	HTTPS enforced in all environments; no HTTP allowed
Authentication	JWT tokens with expiry; bcrypt password hashing
Authorization	Role-based: `citizen` · `admin` · `reviewer` · `government_officer`
Data privacy	No NIC numbers, passport numbers, or biometric data stored in MVP
Document privacy	No uploaded documents stored in MVP
Admin audit	All admin changes recorded in `audit_logs` with before/after state
AI outputs	All AI-generated answers in citizen-facing context must include source citations
Content gating	No service content visible to citizens without `published` status

---

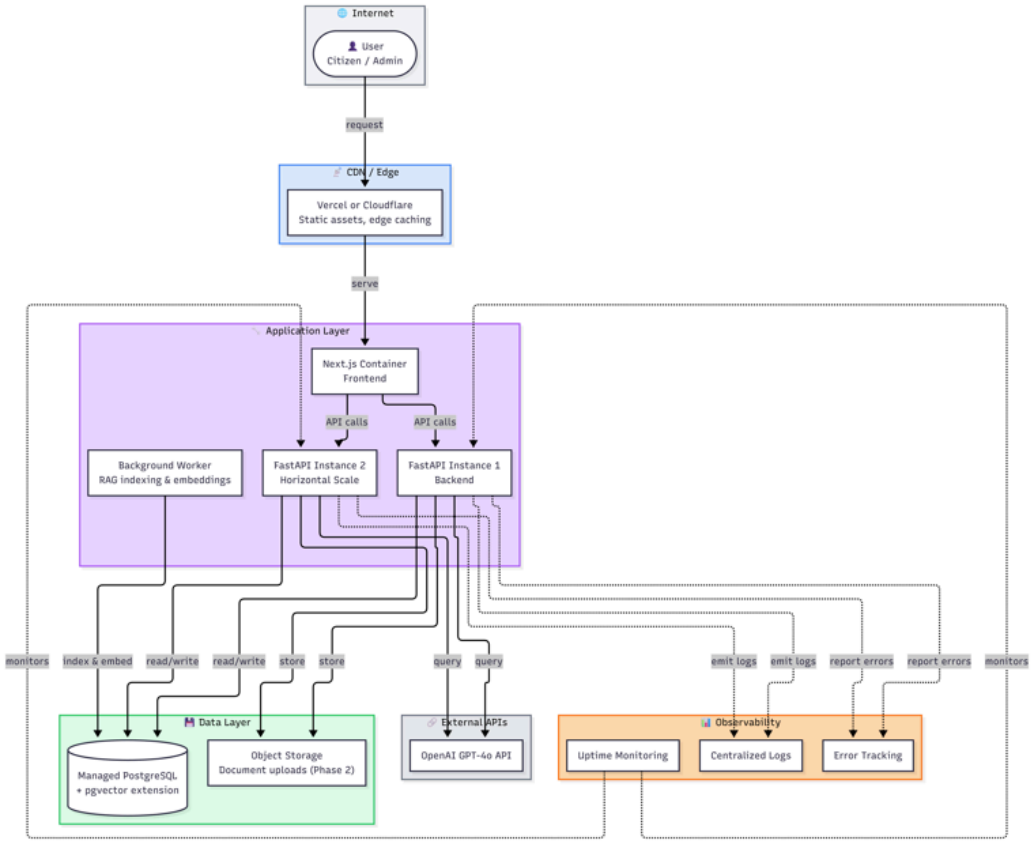
11. Deployment

Development Environment

```
Docker Compose (local)
│
├── Next.js Frontend          localhost:3000
├── FastAPI Backend           localhost:8000
└── PostgreSQL Database       localhost:5432

All services call OpenAI API externally.
```

Production Environment



> ⚠️ **Note:** For a Sri Lankan government-aligned pilot, local or government-approved hosting must be evaluated to comply with data residency requirements.

---

12. Technology Stack

Layer	Technology	Version	Rationale
Frontend	Next.js	14	App Router, SSR, TypeScript support
Frontend language	TypeScript	5.x	Type safety across frontend

UI styling	Tailwind CSS	3.x	Fast iteration, consistent design tokens
Frontend state	React state + Zustand	—	Simple first; Zustand added only if needed
Forms	React Hook Form + Zod	—	Type-safe validation
Backend	FastAPI	0.110+	Fast, typed, excellent AI library support
Backend language	Python	3.12	Best AI/ML ecosystem
ORM	SQLAlchemy + Alembic	2.x	Mature ORM with migration support
Validation	Pydantic	v2	Runtime type safety in FastAPI
Database	PostgreSQL	16	Relational data + pgvector extension
Vector search	pgvector	latest	RAG embeddings alongside relational data
AI orchestration	LangGraph	latest	Stateful, inspectable conversation graphs
LLM	OpenAI GPT-4o	—	Intent classification,

			RAG answers, extraction
OCR	Tesseract	—	Document text extraction (Phase 2)
Auth	JWT + bcrypt	—	Stateless, standard authentication
Containers	Docker + Docker Compose	—	Consistent dev and prod environments
Frontend hosting	Vercel	—	Scalable, low-ops Next.js hosting
Backend hosting	Docker (managed cloud)	—	Horizontal scaling

---

13. MVP Roadmap

Phase Overview

Phase	Focus	Duration
Phase 1 — Foundation	FastAPI, PostgreSQL, Next.js, chat sessions, LangGraph skeleton	2–3 weeks
Phase 2 — Service Engine	Departments, services, admin editor, question engine, condition evaluator	3–4 weeks
Phase 3 — Workflow	Workflow engine, document checklist, user tracking, LLM service matching	3–4 weeks

Phase 4 — RAG & Polish	Source ingestion, pgvector, RAG Q&A, AI extraction assistant, testing	3–4 weeks

Phase 1 — Foundation

- FastAPI project setup with Pydantic, SQLAlchemy, Alembic
- PostgreSQL schema: `chat_sessions`, `chat_messages`
- Next.js project setup with TypeScript and Tailwind
- Basic chat UI with message history
- `POST /chat/sessions` and `POST /chat/sessions/{id}/messages` endpoints
- LangGraph skeleton: `start → detect_intent → fallback`
- LangGraph state saved to `chat_sessions.conversation_state_json`

Phase 2 — Service Engine

- DB schema: `departments`, `services`, `service_versions`, `source_documents`
- Admin UI: create/edit departments, services, versions, source links
- DB schema: `service_questions`, `question_options`
- Question engine: load questions, evaluate conditions, return next question
- Condition evaluator: build and unit test `evaluate_condition(condition, answers)`
- LangGraph nodes: `match_service`, `ask_next_question`, `store_answer`, `check_required_answers`

Phase 3 — Workflow

- DB schema: `workflow_steps`, `required_documents`, `step_documents`, `fees`, `processing_times`
- Workflow engine: apply condition evaluator to steps, documents, fees
- Admin UI: workflow step editor, document requirement editor
- LangGraph node: `generate_workflow`, `explain_workflow`, `save_workflow`
- DB schema: `user_workflows`, `user_workflow_steps`, `user_document_checklist`
- Progress tracker UI: step status, document readiness, percentage complete
- LLM service matching: replace keyword matching with OpenAI-based intent classifier

Phase 4 — RAG & Polish

- Source document ingestion pipeline

- pgvector integration: chunk text, generate embeddings, store vectors
- RAG Q&A: `source_grounded_qa` LangGraph node
- AI content extractor: admin pastes URL → AI suggests structured data → admin reviews
- Feedback module: citizen submissions, admin review queue
- Full testing: condition evaluator unit tests, workflow acceptance tests, API tests

MVP Services

Service	Phase	Department
First-time NIC Application	Phase 3	Department for Registration of Persons
Passport Renewal	Phase 3	Department of Immigration and Emigration
Business Registration (eROC)	Phase 3	Department of Registrar of Companies
Birth Certificate Copy Request	Post-MVP	Registrar General's Department
Driving License Renewal	Post-MVP	Department of Motor Traffic

---

14. Non-Functional Requirements

Category	Requirement	Target
Availability	System uptime	99.5%
Response time	Chat workflow generation	< 3 seconds
Scalability	Backend scaling strategy	Horizontal via containers
Accuracy	Invented fees or requirements	Zero tolerance

Auditability	Published rules must have source URL	100%
Fallback	Behaviour if OpenAI is unavailable	Non-AI guided flow remains functional
Language support	Phase 1	English
Language support	Phase 2	Sinhala and Tamil
Accessibility	Frontend compliance	WCAG 2.1 AA
Privacy	Sensitive data in MVP	Not collected or stored

---

15. Design Decisions

Decision	Rationale
LangGraph from day one	Conversation state management must be stateful from the start. Retrofitting conversation orchestration after the fact is expensive and error-prone.
Rule engine over pure RAG	RAG alone cannot guarantee consistent, traceable government procedure output. Hallucination risk is unacceptable for citizen-facing government guidance.
AI-assisted rule creation	Reduces admin data entry effort while keeping a human reviewer in the loop. AI generates the first draft; admin approves before anything is saved.
Service versioning	Government procedures change. Versioning ensures a citizen who started a workflow in January sees the procedure valid in January, not a modified February version.

Published-status gating	Prevents accidental, untested, or unapproved content from reaching citizens. Explicit admin publish action required.
Modular monolith	Team size and project stage do not justify microservice complexity. Modules are cleanly separated and can be extracted later if needed.
PostgreSQL + pgvector	Keeps relational and vector data in one managed system. Avoids the operational overhead of running a separate vector database.
No sensitive data in MVP	Simplifies data protection compliance, reduces security surface area, and keeps early development focused on the core workflow experience.
